

מבני נתונים 1 - 234218

רטוב 2 – חלק יבש

מגישים:

רועי קסוס - 212240956

מאור גרשמן - 322784166

איתן מיכאלוביץ' - 211878723

תיאור מבנה הנתונים :

מבנה הנתונים שלנו מכיל את תתי מבני הנתונים הבאים :

1. **team_manager** – מבנה UF עם עצים הפוכים, איחוד לפי גודל וכיווץ מסלולים של קבוצות שמשמש בטבלת ערבול על מנת לייצג את הקבוצות השונות בו (לכל קבוצה יש מקום בטבלת הערבול).

2. **record_manager** – רשימה מקושרת דו כיוונית של קבוצות עם מאזן ניטרלי ושתי טבלאות ערבול – אחת לרשימות של קבוצות עם מאזן חיובי ואחת לרשימות קבוצות עם מאזן שלילי. טבלאות הערבול הן לפי מפתח **מאזן**.

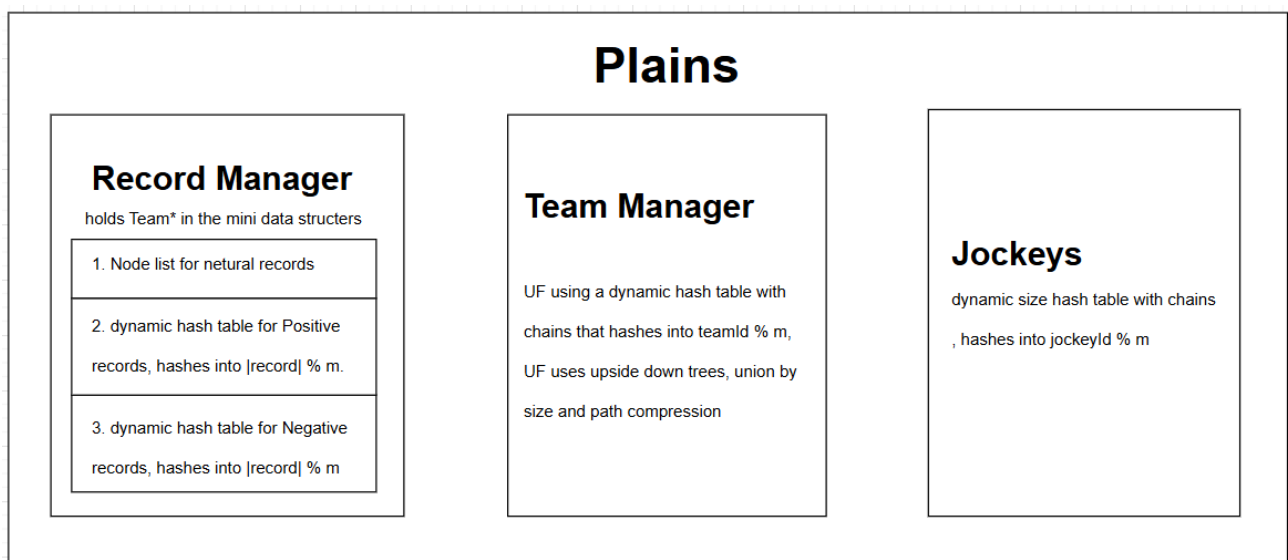
3. **jockeys** – טבלת ערבול של כלל הרוכבים במערכת.

כלל טבלאות הערבול שמוזכרות הם טבלת ערבול בעלת גודל משתנה, שמשמשת בפונקציות הערבול $\text{mod } m$ (שמבטאת את ההנחות שקיימנו על פיזור אחיד כפי שלמדנו בהרצאה) כאשר m הוא גודל הטבלה ומשתמש בשיטת chain hashing. הטבלה מתחילה ריקה, מכפילה את גודלה כאשר מספר האיברים בה הוא $m/2$ ומקטינה את גודלה פי 2 כאשר מספר האיברים בה הוא $m/4$. על כן, נשמר תמיד הכלל של $m = O(n)$ – ועל כן פקטור העומס של הטבלה הוא $O(1)$ לאורך ריצת כל התוכנית. כטענה שראינו בתרגול, מערך דינמי עם תנאי הגדילה והכיווץ הללו, כאשר פעולת ההעתקה כעת מוסיפה את שלב חישוב hash מחדש שהיא פעולה של $O(1)$ שומר על סיבוכיות משוערכת של $O(1)$ עם הפעולות שלו ולכן פעולות ההכנסה, החיפוש וההוצאה בטבלת הערבול ישמרו על סיבוכיות זמן ריצה של $O(a)$ בממוצע על הקלט כאשר a הוא פקטור העומס שהסברנו לעיל שהוא לאורך כל זמן הריצה $O(1)$.

העצמים שאנו משתמשים בהם :

1. **קבוצה** - כל קבוצה מחזיקה את הגודל שלה (כמה קבוצות קיימות בתוכה (אם היא השורש)), מצביע לאב, את המאזן שלה ואת מזהה הקבוצה שלה. בקוד זה מפוצל לסטרקטים של `RootData` ו `NonRootData`. בנוסף כל קבוצה מחזיקה מצביע ל `Node` שלה ב `record_manager` לטובת ייעול פעולות בהמשך.
2. **רוכב** – מחזיק מצביע לקבוצה שאליה הצטרף לראשונה ומאזן אישי.

תיאור גרפי של מבנה הנתונים :



ניתוח סיבוכיות מקום של מבני הנתונים :

בעבור כל קבוצה, קיימים 3 הקצאות זיכרון בכל רגע נתון – הקבוצה עצמה (שגם משמשת Noden בעץ ההפוך), "המקום שלה" בטבלת הערבול במבנה UF, "המקום שלה" בrecord manager באחת משלושת האפשרויות. נשים לב, שלאורך כל ריצת התוכנית, גודל טבלת הערבול של team manager היא $O(m)$ כאשר m הוא מספר קבוצות הרוכבים שנוספו למערכת מכיוון שגודלה משתנה דינאמית בהתאם למספר הקבוצות כפי שמתואר ושומר על גודל $O(m)$ כך. בנוסף, נשים לב שכל אחד ממבני הנתונים בתוך record manager הוא בגודל $O(m)$ מאותה הסיבה (גם גודל הרשימה המקושרת לא יעלה על $O(m)$ מכיוון שלכל היותר כל הקבוצות נמצאות בו). על כך בהערכה מחמירה כי קבוצה יכולה להיות בכל שלושת התת מבנים של record manager (על אף שזה לא אפשרי בקוד), נקבל כי סיבוכיות המקום בעבור כל קבוצה היא $5O(m)$ כלומר $O(m)$.

בעבור כל רוכב, קיים העצם של הרוכב וגודל טבלת הערבול של הרוכבים, נשים לב שמאותו הסבר של הקבוצות ואופן הגדלת והקטנת טבלת הערבול גודל טבלת הערבול לאורך כל הריצה הוא $O(n)$ כאשר n הוא מספר הרוכבים במערכת.

סה"כ נקבל כי קיימות 5 תתי מבני נתונים, 3 טבלאות ערבול ורשימה מקושרת בעבור הקבוצות, וטבלת ערבול בעבור הרוכבים, כולם שומרים על סיבוכיות מקום לינארית בעבור מספר הקבוצות שנוספו והרוכבים בהתאמה, ולכל רוכב וכל קבוצה מוקצה העצם שלו עצמו. על כן נקבל כי בסה"כ סיבוכיות המקום של מבנה הנתונים הוא :

$$O(m) + O(n) + 4O(m) + O(n) = 5O(m) + 2O(n) = O(m) + O(n).$$

פעולות שנדרשנו לממש וניתוח סיבוכיות זמן הריצה שלהם :

1. **plains_t()** – יוצר את כל תתי מבני הנתונים ריקים ועם ערכים דיפולטיביים, טבלת ערבול התחלתית מוקצית בגודל 10 (טנטטיבי לחלוטין). כל שמבוצע הוא קריאות לבנאים עם ערכים דיפולטיים, דבר שמתבצע בסיבוכיות זמן $O(1)$ ועל כן סיבוכיות הזמן של הפעולה היא $O(1)$.

2. **virtual ~plains_t()** – ההורס יקרא לכל ההורסים של תתי המבנים, שבתורם יבצעו מחיקה של :

א. **jockeys** – כל עצמי הרוכבים וטבלת הערבול שהחזיקה אותם – מעבר לינארי על כל העצמים בטבלה ולאחר מכן מחיקת הטבלה עצמה – $O(n)$.

ב. **Team manager** – כל עצמי הקבוצות וטבלת הערבול שהחזיקה אותם – מעבר לינארי על כל העצמים בטבלה ולאחר מכן מחיקת הטבלה עצמה – $O(m)$.

ג. **Record manager** – מחיקת 2 טבלאות הערבול של המאזנים חיובי ושלילי + מחיקת הרשימה המקושרת של הקבוצות עם מאזן ניטרלי – $O(m)$ במקרה הגרוע בו לכל הקבוצות יש מאזן ניטרלי.

סה"כ נקבל כי כלל פעולות המחיקה רצות בסיבוכיות זמן ריצה של :

$$O(n) + O(m) + O(m) = O(n) + 2O(m) = O(n) + O(m).$$

3. **add_team (int teamId)** – ראשית, נבצע חיפוש בטבלת הערבול של הקבוצות האם הקבוצה כבר קיימת (במובן חזק, כלומר, קבוצה שאוחדה פעם – נחשבת לא קיימת (יש לה מצביע לאבא)), אם קיימת נחזיר שגיאה. אחרת, נקצה אותה דינאמית ונבצע הכנסה לטבלת הערבול של הקבוצות. בנוסף, נבצע הכנסה של recordNode שתואם לקבוצה (עם המזהה) לתוך רשימת הקבוצות עם מאזן ניטרלי, ונעדכן את עצם הקבוצה עם מצביע לעצם זה. כטענה מהתרגול, פעולת הכנסה למערך דינאמי היא פעולה של $O(1)$ בשערוך עם עצמה, ופעולות ההכנסה, החיפוש וההוצאה לטבלת ערבול בשיטת הרשימות היא $O(a)$ בממוצע על הקלט, הסברנו לעיל ש $O(a) = O(1)$ לכל טבלאות הערבול בתוכנית שלנו ולכל זמן ריצת התוכנית ועל כן נקבל שסה"כ ביצענו פעולת חיפוש והכנסה לטבלת הערבול הדינאמית פלוס הכנסה לרשימה מקושרת דו כיוונית שהיא פעולה של $O(1)$ כטענה מההרצאה ועל כן סיבוכיות זמן הריצה של הפעולה היא $O(1)$ **בממוצע על הקלט משוערך עם עצמו.**

4. **add_jockey(int jockeyId, int teamId)** – ראשית, נחפש את הרוכב בטבלת הרוכבים, אם קיים – נחזיר שגיאה. אחרת, נחפש בטבלת הערבול של הקבוצות את מזהה הקבוצות ונקבל את המצביע אליו, נציין בפרט שזוהי **לא** פעולת find, אלא פשוט חיפוש רגיל בטבלת הערבול, כעת נקצה את העצם של הרוכב ונכניס אותו לטבלת הערבול של הרוכבים. בדומה לסעיף לעיל, כל שביצענו הוא שני פעולות של חיפוש ופעולת הכנסה לטבלת ערבול דינאמית, ומאותם תירוצים סיבוכיות זמן הריצה היא $O(1)$ **בממוצע על הקלט משוערך עם עצמו.**

5. **update_match(int victoriousJockeyId, int losingJockeyId)** – ראשית, נבצע חיפוש של שני הרוכבים בטבלת הערבול שלנו – אם אינם קיימים, נחזיר שגיאה. אחרת, קיבלנו את המצביעים אליהם, ניגש למצביעי "הקבוצות המקוריות" שלהם שהם מחזיקים ונבצע שתי פעולות find של UF על מנת לבדוק האם הם באותה קבוצה, אם נקבל שהם באותה קבוצה, נחזיר שגיאה. אחרת, נוסיף לקבוצה המנצחת (השורש שלה) למאזן אחד, נחסיר מהקבוצה המפסידה (לשורש שלה) למאזן אחד – נבצע הוצאה סימבולית (ללא מחיקת הזיכרון) מהרשימות המקושרות שבהם הקבוצות נמצאות (בין אם בניטרלי או בין אם בחיפוש המאזן לפני העדכון בטבלת הערבול ואז בהוצאה מהרשימה שקשורה במפתח הני"ל), ואז נבצע הכנסה סימבולית (ללא הקצאה חדשה, ולסוף הרשימה המקושרת) של אותו הנוד למקומות הנכונים באמצעות הכנסה לטבלת הערבול עם המפתח של המאזן החדש. לבסוף נבצע עדכון גם למאזן האישי של כל רוכב. סה"כ ביצענו : 2 חיפושים בטבלת הערבול של הרוכבים, שני פעולות find במבנה UF עם הקבוצות המקוריות שהרוכבים החזיקו, 2 חיפושים בטבלת הערבול של המאזנים, 2 הוצאות **כאשר המיקום ידוע** (אנחנו שומרים מצביע לnode בטבלה הרלוונטית בקבוצה כל הזמן, בהתחלה הוא מתחיל בקבוצה הניטרלית ולאחר מכן במעברים בין המאזנים זה רק משחק מצביעים כיוון שתמיד נכנס לסוף, ואם יוצא שוב יש לנו את מקומו בתוך הרשימה הרלוונטית בrecordNode כי העצם לעולם לא השתנה, רק הוכנס לרשימה מקושרת אחרת ומצביעיו קדימה ואחורה התעדכנו בהתאם) ושני הכנסות לרשימה מקושרת רלוונטית. כטענות מההרצאה על פעולת find במבנה UF עם עצים הפוכים, איחוד לפי

גודל וכיווץ מסלולים, החיפוש בטבלאות הערבול הדינמיות שלנו כפי שהסברנו לעיל, והוצאה והכנסה מרשימה מקושרת דו כיוונית כאשר מיקום ההוצאה **ידוע**, נקבל כי סה"כ סיבוכיות זמן הריצה של הפעולה היא (חיבור לפי סדר ההסבר):

$$2O(1) + 2O(\log^* m) + 2O(1) + 2O(1) + 2O(1) = O(\log^* m).$$

הסיבוכיות $O(\log^* m)$ היא בממוצע על הקלט ומשוערכת עם הפעולות `merge_teams` ו-`unite by record` ולא במקרה הגרוע, מכיוון שאנחנו לוקחים בחשבון פעולות בטבלת הערבול שהן $O(1)$ בממוצע על הקלט ולא במקרה הגרוע, וטוענים כי פעולת `find` היא בסיבוכיות זמן ריצה של $O(\log^* m)$ במבנה ה-UF שלנו וזוהי טענה שלוקחת בחשבון שערך עם פעולת Union שמתרחשת בשני הפונקציות האחרות שהוזכרו לעיל **ולכן כל ניתוח הסיבוכיות שבוצע למעלה הוא בממוצע על הקלט ומשוערך עם הפעולות האחרות ולא במקרה הגרוע.**

6. `merge_teams(int teamId1, int teamId2)` – נבצע איחוד לפי גודל רגיל לחלוטין כפי שנלמד בכיתה, וכעת, על מנת לעמוד בדרישות התרגיל לביצוע איחוד לפי מאזן, כל שנעשה הוא לפני שנבצע את האיחוד, נזכור את מזהה הקבוצה מבין השניים שלה המאזן הטוב יותר, ונעדכן את מזהה הקבוצה של הקבוצה הגדולה יותר להיות מזהה הקבוצה של הקבוצה הטובה יותר. במידת הצורך (הקבוצה הטובה יותר היא הקטנה יותר) – נעדכן את המידע בטבלת ערבול, ונדאג לכך שמקום הקבוצה הגדולה בטבלת הערבול כעת יחזיק את הערכים של הקבוצה הקטנה (שכן ביצענו איחוד לפי גודל), ומקום הקבוצה הקטנה (הטובה) יחזיק את הערכים של הקבוצה הגדולה (שכן האיחוד דורש שהקבוצה הטובה תהיה הקבוצה האמיתית (שהיא החיבור לקבוצה הגדולה מביניהם ב-UF רגיל). באופן הזה, טבלת הערבול שומרת על נכונותה, ויתר על כך, הקבוצה שמתאחדת עם אחרת נשארת בטבלת הערבול, וכך לא ניתן להוסיף קבוצה עם מזהה שכבר אוחד לקבוצה אחרת בעבר. נוסף על כך, נבצע את העדכונים הנדרשים במאזני הקבוצות כפי שמתואר מעלה (הוצאות והכנסות מהרשימות המקושרות שבתוך טבלת הערבול). חשוב לציין כי בגלל איך שאנו מבצעים את פעולת האיחוד, והעובדה שאנחנו דואגים לכך שמזהה הקבוצה מתעדכן, כל "קבוצה מקורית" שיש לה מצביע לאב היא קבוצה לא תקינה יותר, וכל קבוצה שאין לה, היא קבוצה ששמרה על תקינותה (איחדו לתוכה) ולה יש מידע נוסף רלוונטי. כל שהוספנו לפעולה הרגילה של איחוד במבנה UF שנלמדה בהרצאה היא שינוי מזהה לקבוצה הגדולה, עדכון ערכים בטבלת הערבול הדינמית של הקבוצות, ועדכון רשימות מקושרות בטבלאות הערבול של המאזנים, כולן פעולות $O(1)$, ועל כן סיבוכיות זמן הריצה המשוערכת של הפעולה לא נפגעה **אך התווסף המיצוע על הקלט מכיוון שאנו משתמשים בטבלת ערבול ועל כן סיבוכיות זמן הריצה של הפעולה היא $O(\log^* m)$ היא בממוצע על הקלט ומשוערכת עם הפעולות `merge_teams` ו-`update match`, שוב מכיוון שפעולת `union` משוערכת עם פעולת `find` שמתבצעת ב-`update match`.**

7. **unite_by_record(int record)** – ראשית נבצע חיפוש בטבלאות הערבול של המאזנים, אחד עם הערך השלילי ואחד עם החיובי, ונקבל 2 רשימות מקושרות עם כל הקבוצות במאזנים אלו. מכיוון שאנו שומרים משתנה size ומעדכנים אותו ברשימה המקושרת ניתן לדעת ב- $O(1)$ את גודלה, ככל שגודלה של אחת משני הרשימות המקושרות איננו 1, נחזיר שגיאה. אחרת, נקח את שני הקבוצות מהרשימות המקושרות, כעת אנו יודעים שהן היחידות עם המאזן הנ"ל ונבצע עליהם פעולת merge_teams כפי שהוסברה לעיל. סה"כ ביצענו 2 חיפושים בטבלאות ערבול לפי מפתח המאזן, וקריאה לפעולה merge_teams שלנו, ועל כן סיבוכיות הפעולה היא $O(\log^* m)$ **היא בממוצע על הקלט ומשוערכת עם הפעולות merge_teams וupdate match**.

8. **get_jockey_record(int jockeyId)** – נבצע חיפוש בטבלת הערבול של הרוכבים, אם הרוכב לא קיים נחזיר שגיאה, אחרת נחזיר את המאזן האישי שלו שאנו דואגים לעדכן בעצם שלו לאורך הריצה. זוהי פעולת חיפוש בטבלת ערבול ועל כן סיבוכיות זמן הריצה היא $O(1)$ **בממוצע על הקלט**.

9. **get_team_record(int teamId)** – נבצע חיפוש בטבלת הערבול של הקבוצות שמנוהלת ע"י team_manager, אם הקבוצה לא קיימת נחזיר שגיאה. אחרת, נבדוק אם לקבוצה קיים מצביע לאב, אם קיים, הקבוצה היא לא שורש, כלומר עברה איחודים ומזהה הקבוצה שלה "נבלע" לקבוצה הטובה יותר במהלך ריצת התוכנית. לכן קבוצה תקינה אמ"מ היא שורש כלומר אין לה מצביע לאב ובמקרה כזה נחזיר את record שמעודכן בה ושמור בה. אחרת, נחזיר שגיאה כי זוהי קבוצה לא תקינה. זוהי פעולת חיפוש על טבלת ערבול דינמית ועל כן סיבוכיות זמן הריצה היא $O(1)$ **בממוצע על הקלט**.